

Exception Handling Abuse in C# with an Example

In this article, I am going to discuss **Exception Handling Abuse in C#** and then we will see how to prevent Exception Handling Abuse in C# with one example. Please read our previous article where we discussed the [Inner Exception in C#](#) with an example.

Exception Handling Abuse in C#:

Exceptions are nothing but classes that terminate the program execution abnormally when runtime errors occur during the execution of a program. For example, when an application is executing a database query, due to some reason the database connection is lost, then we will get an `SQLException` runtime error. Suppose, we are trying to fetch the data from a text file, and that text file does not exist, then we will get a `File Not Found Exception`. So, Exception handling is generally used to handle these scenarios.

But sometimes, as a programmer, we are using exception handling mechanisms to implement programming logic which is bad, and this is called Exception Handling Abuse in C#.

Example: Here we are using exception handling to implement logical flow:

In the below example, we are using the Exception Handling Mechanism for Implementing the business logic. That is we are using Exception Handling for Handling the Input Format, Divide Operations, and Handling the Max and Min Length. This is nothing but Exception Handling. These things we can manage without using Exception Handling.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        public static void Main()
        {
            try
            {
                //Convert.ToInt32() can throw FormatException, if the entered
                //cannot be converted to integer. So use int.TryParse() instead
                Console.WriteLine("Please enter First Number");
                int FirstNumber = Convert.ToInt32(Console.ReadLine());
                Console.WriteLine("Please enter Second Number");
                int SecondNumber = Convert.ToInt32(Console.ReadLine());
                int Result = FirstNumber / SecondNumber;
                Console.WriteLine($"Result = {Result}");
            }
            catch (FormatException)
            {
                Console.WriteLine("Only numbers are allowed!");
            }
            catch (OverflowException)
            {
            }
        }
    }
}
```

```

        Console.WriteLine($"Only numbers between {Int32.MinValue} &
{Int32.MaxValue} are allowed");
    }
    catch (DivideByZeroException)
    {
        Console.WriteLine("Second Number cannot be zero");
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
    Console.ReadLine();
}
}
}

```

Preventing Exception Handling Abuse:

Let's rewrite the same example that doesn't use exception handling to control the program's logical flow. In the below example, we are using tryParse for Handling the Input Format, If Else condition to handle the Divide Operations, and Max and Min Length.

```

using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        public static void Main()
        {
            try
            {
                Console.WriteLine("Please Enter First Number");
                int FirstNumber;
                //int.TryParse() will not throw an exception, instead returns
                //if the entered value cannot be converted to integer
                bool IsValidFirstNumber = int.TryParse(Console.ReadLine(), out
FirstNumber);
                if (IsValidFirstNumber)
                {
                    Console.WriteLine("Please Enter Second Number");
                    int SecondNumber;
                    bool isValidSNO = int.TryParse(Console.ReadLine(), out Sec
                    if (isValidSNO && SecondNumber != 0)
                    {
                        int Result = FirstNumber / SecondNumber;
                        Console.WriteLine($"Result = {Result}");
                    }
                    else
                    {
                        //Check if the second number is zero, print a friendly

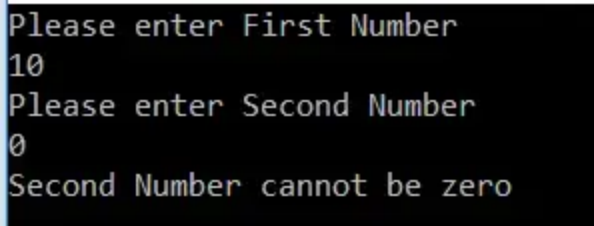
```

```

        //message instead of allowing DivideByZeroException ex
        //to be thrown.
        if (isValidSNO && SecondNumber == 0)
        {
            Console.WriteLine("Second Number cannot be zero");
        }
        else
        {
            Console.WriteLine($"Only numbers between {Int32.Mi
{Int32.MaxValue} are allowed");
        }
    }
}
else
{
    Console.WriteLine("Only numbers between {Int32.MinValue} &
{Int32.MaxValue} are allowed");
}
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}
Console.ReadLine();
}
}
}
}

```

Output:



```

Please enter First Number
10
Please enter Second Number
0
Second Number cannot be zero

```

In the next article, I am going to discuss the [Delegates in C#](#) with Examples. Here, in this article, I try to explain **Exception Handling Abuse** with Examples. I would like to have your feedback. Please post your feedback, question, or comments about this article.